

Reality Gap Analysis: Domain Randomization for Sim-to-Real Transfer

Ludovica Astolfi Francesco Borlandi Luca Bosso Gabriele Triscari
Politecnico di Torino
Turin, Italy

{s360569, s364074, s362957, s360858}@studenti.polito.it

Abstract

This report investigates the problem of learning control policies through Reinforcement Learning (RL) in simulation and studying Sim-to-Real transfer within a Sim-to-Sim context. Initially, we evaluate the REINFORCE and Actor-Critic algorithms in the Hopper environment, demonstrating that Actor-Critic halves the episodes required for convergence compared to REINFORCE. Subsequently, in a PushTask robotic manipulation environment, we analyze how the off-policy SAC algorithm significantly outperforms the on-policy PPO algorithm, achieving a 74.00% success rate compared to PPO's 30.00%. To address the reality gap caused by a mass discrepancy, we implement Uniform (UDR) and Adaptive Domain Randomization (ADR) strategies. UDR requires rigorous boundary tuning: an optimal range of [0.5, 2.5] kg achieves a 100.00% success rate. ADR dynamically automates this adaptation, also guaranteeing a 100.00% success rate. We conclude that, although Domain Randomization efficiently bridges the reality gap, extreme variance induces brute-force strategies.

Repository: github.com/lboxx5-lang/faiml-rl-project

1. Introduction

1.1. Project Context

Reinforcement Learning (RL) is an approach in which an agent aims to maximize a scalar reward through trial-and-error interactions with the environment.

However, a crucial barrier is the Sim2Real problem: transferring skills developed during simulation to real life often proves more difficult than expected due to discrepancies known as the “reality gap”.

In this work, we manually introduce a discrepancy between the training (source) and testing (target) domains to simulate the reality gap. To overcome this, we apply Domain Randomization, a strategy used to train robust policies capable of handling such environmental variations.

1.2. Literature Foundations

The foundational approach for continuous control in our experiments is based on the REINFORCE algorithm and the Actor-Critic architecture [1]. As these foundational models present some limitations in terms of stability and efficiency, two advanced algorithms that have recently been studied are implemented to improve these earlier methods: Proximal Policy Optimization (PPO) [8] and Soft Actor-Critic (SAC) [7].

REINFORCE is a Policy Gradient learning algorithm, which means that it updates the policy parameter by a quantity proportional to the product of a return and the gradient of the probability of taking the action. REINFORCE uses the complete return from time t , which includes all future rewards up until the end of the episode. This approach inherently leads to an extremely high variance and, as a consequence, results in a significantly unstable training process. To reduce the variance and accelerate the algorithm, we introduce a state-dependent baseline.

Although a state-dependent baseline helps in reducing variance, it evaluates the state prior to the transition's action. To solve this problem and to assess actions, we introduce the Actor-Critic architecture, where the state-value function is also applied to the second state of the transition.

PPO is an *on-policy* algorithm, which means that it only learns from current actions. It has an extremely stable training phase due to a bound applied to its objective function, called *clipping*. This mechanism prevents the algorithm from significantly changing the previous policy, avoiding destructive policy updates such that agents do not forget already learned optimal behaviors.

On the other hand, SAC is an *off-policy* algorithm. Off-policy algorithms are generally more efficient from the sampling point of view with respect to on-policy algorithms, as they reuse past experiences stored in a replay buffer rather than discard them after each policy update. SAC modifies the objective function to simultaneously maximize the expected reward and the entropy of the policy. Maximizing the

entropy means encouraging a wide exploration, thus avoiding early convergence of the algorithm to sub-optimal actions and achieving a more robust policy.

An additional challenge in robotic continuous control is represented by the transfer of the learned policy from simulated environment to the real world, a process known as *Sim-to-Real transfer* [4]. Due to evident discrepancies between real world physics and the simulation, policies trained in ideal virtual environments tend to fail when applied to real hardware. A valid, well-established strategy to solve this issue is *Domain Randomization* (Uniform and Automatic) [5], a technique that consists of changing physical parameters during the training phase, forcing the agent to implement a generalizable policy, flexible to perturbations.

2. Methodology

2.1. Environments and Reality Gap

The project uses two simulated environments to validate our algorithmic foundations before addressing PandaPush task, we first evaluate standard policy gradient methods on the simpler Hopper environment. The first one is the Hopper environment, a one-legged robot model, whose objective is to maximize the horizontal speed without falling. The observation space is continuous and is defined as a Box(-Inf, Inf, (11,), float64). It represents positions and speed of the robot. The action space is also continuous and it is defined as Box(-1, 1, (3,), float32). It defines the torques applied to the joints.

The second environment is the PushTask, which is a robotic manipulation task in which the agent’s objective is to push a cube towards a goal position. In order to simulate the reality gap, we used a source domain in which the cube has a mass of 1kg, and a target domain in which a 5kg mass cube is used, introducing a significant physical discrepancy.

2.2. Implementation Choices and Hyperparameters

Regarding Hopper, we analyzed REINFORCE and Actor-Critic. The neural networks (Actor and Critic) use standard architectures with 64-unit hidden layers and *Tanh* activation function. Weights are initialized with a standard deviation equal to 1. The Actor returns a Gaussian distribution (with initial σ equal to 0.5), whereas the Critic estimates the state value $V(s)$. We used the Adam optimizer with a learning rate of 10^{-3} and $\gamma = 0.99$. The network update happens once per episode. In the Actor-Critic, the total loss is the unweighted sum of actor and critic losses.

The REINFORCE analysis is tested over constant baseline $b \in \{0, 20, 100, 300\}$ for 10000 episodes. Extended tests over Actor-Critic (500000 steps) confirmed the achievement of stable returns close to 1800.

For the PushTask environment, we evaluated the PPO and SAC algorithms training both models for 500000 steps. For

PPO, we used a learning rate of $3 \cdot 10^{-4}$, a batch size of 64 and a discount factor $\gamma = 0.99$. For SAC, we set a learning rate of $3 \cdot 10^{-4}$, a batch size of 256, $\gamma = 0.95$ and a soft-update coefficient $\tau = 0.01$.

Regarding the Domain Randomization techniques, the models were also trained for 500000 steps. Specifically for UDR, we tested and compared two distinct ranges for mass randomization: a broader range of $[0.5, 4.0]$ kg and a narrower one of $[0.5, 2.5]$ kg.

For the ADR strategy, we initialized the environment at a deterministic point with a mass of 1.0 kg. Rather than using independent boundary sampling, we implemented a symmetric adaptation rule to maximize sample efficiency. By evaluating a global success rate buffer, the active bounds are symmetrically expanded by a step of 0.25 if the success rate exceeds 0.8, and shrunk towards the center if it falls below 0.2. Mass values are then sampled uniformly within these dynamically adapting bounds, strictly restricted between 0.5 kg and 4.0 kg.

2.3. Evaluation Metrics

To compare performance, we analyzed the following metrics:

Metrics for Hopper environment

- *Final Ret (mean $\pm$ std)* (Reward): It indicates the average cumulative score achieved by the agent at the end of the training, computed with respect to 3 random seeds. In particular, the standard deviation represents the reproducibility of the algorithm.
- *Intra-run Std*: It measures how much the reward oscillates within a single execution.
- *Ep to Target (>100)*: It indicates the number of episodes required for the algorithm to exceed a predefined reward threshold (equal to 100). Specifically, we use this metric to measure the initial speed of convergence.
- *Time/Ep (ms)*: It measures the average time taken to complete a single episode and indicates computational efficiency.

Metrics for PushTask environment

- *Success Rate(%)*: it indicates the percentage of task completion over 50 test episodes.
- *Mean Return*: it represents the average score obtained during the test phase.

3. Experimental Results

3.1. Basic Algorithms: Hopper

In this section, we present the quantitative results obtained during the training phase of the baseline algorithms in the Hopper environment. Initially, we evaluate the impact of introducing a constant baseline on the REINFORCE algorithm. Later on, we compare the resulting best performing

REINFORCE configuration with the Actor-Critic architecture, in order to underline learning differences. The data

Table 1. REINFORCE Baseline Comparison

Baseline	Final Ret	Intra-run Std	Time/Ep (ms)
b=0	267.6 $\pm$ 198.5	105.9	25.00
b=20	114.9 $\pm$ 85.7	35.1	21.19
b=100	18.9 $\pm$ 25.1	7.6	2.18
b=300	1.4 $\pm$ 1.7	1.4	1.87

in the above table allow us to compare the performance of the REINFORCE algorithm across various baseline values. The vanilla configuration (b=0) achieves the highest final average reward (267.6), but it also shows a significant instability across different seeds (± 198.5). Changing the baseline to b=20 causes the final return to fall to 114.9, whereas the time per episodes remains consistent with the vanilla configuration. On the other hand, when baseline value increases (b=100 and b=300) the algorithm fails, achieving returns close to zero (18.9 and 1.4) and registering a sharp decline in the time for episode (2.18 ms and 1.87 ms).

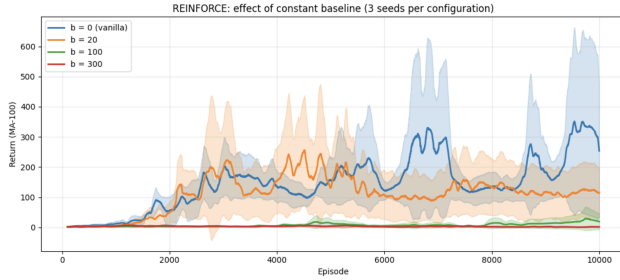


Figure 1. Learning curves of REINFORCE under different constant baseline configurations ($b \in \{0, 20, 100, 300\}$).

As observed in the learning curves in the graph above, the decline in performance is directly proportional to the increase in the baseline. The curves for b=100 and b=300 remain flat at near zero values throughout the entire training. The significant reduction in time per episode indicates that the agent drops almost immediately, leading to premature episodes termination.

Table 2. REINFORCE vs Actor-Critic Comparison

Baseline	Final Ret	Intra-run Std	Ep to Target(>100)	Time/Ep (ms)
REINFORCE b=0	267.6 $\pm$ 198.5	105.9	2754	25.00
Actor-Critic	284.5 $\pm$ 7.0	42.3	1376	20.79

From the table above, it is noticeable that the Actor-Critic architecture (AC) is significantly faster: it overcomes the return threshold after just 1376 episodes. In contrast,

the vanilla configuration takes 2754 episodes to achieve the same goal, so it takes exactly double the episodes. The average time per episode remains consistent (20.79 ms for AC against 25.00 ms for REINFORCE).

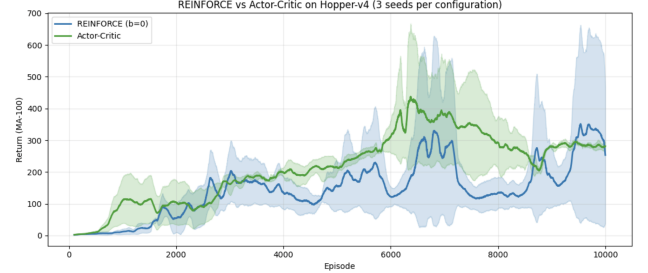


Figure 2. Learning curves of REINFORCE (b=0) and Actor-Critic

As illustrated in the graph above, the main difference between the Actor-Critic architecture and REINFORCE (b=0) lies in the standard deviation throughout the entire process. The standard deviation bands in the learning curves confirm that AC presents much greater stability than the REINFORCE algorithm.

3.2. Advanced algorithms and Sim-to-Real: Push-Task

Now, we present quantitative results obtained in the PushTask environment. Initially, we compare the PPO and SAC performances, then we analyze performance decline generating by the transfer of the policy from simulation to reality. In the end, we evaluate efficiency of introducing Domain Randomization techniques.

Table 3. PPO vs SAC Comparison

Algorithm	Configuration	Success Rate	Mean Return
PPO	Source $\rightarrow$ Source	30.00%	-3.056
SAC	Source $\rightarrow$ Source	74.00%	-1.647

The table above underlines a clear difference between the two algorithms. Looking at the data, we can gather that PPO (with a success rate of 30.00% and a mean return of -3.056) is less efficient for this specific task than SAC (with a success rate of 74.00% and a mean return of -1.647). As a consequence, we selected the SAC algorithm for all the following tasks.

Table 4. SAC Configurations Comparison

Configuration	Success Rate	Mean Return
Source $\rightarrow$ Source	74.00%	-1.647
Source $\rightarrow$ Target	58.00%	-2.149
Target $\rightarrow$ Target	100.00%	-0.392

Considering the policy trained and tested on the source domain as a baseline, it is noticeable that the success rate slightly falls when testing the same policy on the target domain (a 58.00% of success rate). This value represents our lower bound.

On the other hand, when training and testing the policy on the target domain, the success rate increases significantly, reaching 100.00%. This value represents our upper bound.

Table 5. SAC with Domain Randomization techniques

Configuration	Success Rate	Mean Return	Range(kg)
Source (Random) $\rightarrow$ Target	42.00%	-2.821	[0.5, 4.0]
Source (Random) $\rightarrow$ Target	100.00%	-0.479	[0.5, 2.5]
Source (Random) $\rightarrow$ Target	100.00%	-0.437	[0.5, 10.0]
Source (Adaptive) $\rightarrow$ Target	100.00%	-0.543	-

As shown in the table, UDR results vary depending on the applied range. Using a range of [0.5, 4.0] kg, the success rate is 42%, with a mean return of -2.821. Narrowing the range to [0.5, 2.5]kg, the configuration achieves a success rate of 100.00% and a mean return of -0.479. Conversely, expanding the range to [0.5, 10.0]kg also yields a success rate of 100%. Similarly, ADR obtains a success rate of 100.00%, with a mean return of -0.543.

4. Discussion

4.1. Result Analysis

The results obtained with the different REINFORCE configurations fully confirm our expectations regarding the practical functioning of the baseline. We knew that subtracting constant values that was too high, such as $b=100$ or $b=100$, would literally bias the agent. Since the empirical return subtracted from such a high baseline is almost always negative, the algorithm ends up interpreting every action, even the correct ones, as mistakes to avoid. We therefore deduce that the agent, receiving only negative feedback, stops exploring almost immediately and eventually gives up; this perfectly explain why our learning curves show very short and an immediate collapse to zero return. Conversely, the standard configuration ($b=0$) learns, but it shows large oscillations because it lacks an internal baseline for comparison.

Observing the learning curves of the Actor-Critic architecture, as expected, it is noticeable that AC proves to be better

in stability and speed compared to the REINFORCE algorithm. While the REINFORCE algorithm needs to complete an entire episode to understand if the sequence of actions was useful to achieve the goal, Actor-Critic evaluates and corrects its moves at every single step. As a consequence, while the REINFORCE algorithm accumulates errors, AC corrects them in real time.

These characteristics lead AC to achieve the target taking less time (half the episodes) and, in particular, showing a more stable trend.

Regarding the PushTask environment, the difference in performance between PPO and SAC is not surprising and is entirely consistent with the complexity of the task. As an on-policy algorithm, PPO fails to construct a robust strategy within the 500,000 available steps because it inherently discards data after each update. Furthermore, to ensure a fair comparison, we conducted additional hyperparameter tuning on the PPO baseline; despite these efforts, the algorithm never exceeded a peak success rate of 30.00%. This confirms that its underperformance is not a consequence of a suboptimal configuration, but rather a fundamental limitation of its on-policy sample efficiency. On the other hand, SAC’s success rate is significantly higher because its off-policy nature allows it to store and reuse past experiences via a replay buffer. These results confirm that, for continuous robotic tasks that require extensive exploration, an off-policy approach that reuses samples is a much more efficient choice.

Continuing with the analysis of the "Reality Gap", the results are coherent and logical if we consider the fundamental assumptions of the environments: in the source domain the cube weights 1kg, whereas in the target domain it weights 5kg.

Considering what we just established, the result obtained by training and testing the agent in the source domain (74.00% success rate) means that the agent learned to manage the lighter cube. However, moving a light 1kg cube means that even a small, but strong move has enormous consequences, making it much more difficult to control. This explains why the success rate does not exceed 74.00%.

When we take the same policy and test it in the target domain the success rate falls to 58.00%. We expected this result due to the direct effect of Reality Gap: the agent tries to apply forces calibrated for a lighter cube onto a 5kg cube. As a consequence, these forces are almost insufficient and cannot move the heavy cube effectively, resulting in a lower success rate.

Before proceeding with the Domain Randomization analysis, it is worth mentioning an additional hyperparameter optimization phase that we conducted.

Table 6. Mass variation in optimized configuration

Mass	Success Rate	Mean Return
5	98.00%	-0.402
10	98.00%	-0.604
30	80.00%	-1.480
50	48.00%	-2.808
100	34.00%	-2.750

By increasing the batch size to 512 and reducing the soft-update coefficient $\tau = 0.005$, we obtained a SAC policy able to achieve a success rate of 98.00% in the Source $\rightarrow$ Target configuration (5kg), without Domain Randomization.

To investigate this anomaly, we executed a stress test, gradually increasing the target mass to 10, 30, 50 and 100 kg. The resulting policy maintained a success rate equal to 98.00% up to 10kg, but then it dropped drastically to 34.00% with a 100kg cube. This behavior indicated that the optimized policy had not solved the Sim-to-Real gap through physical adaptation. In contrast, it had learned a brute-force strategy, applying maximum torques regardless of the cube’s mass. Even if this approach is efficient on lighter masses, it fails on heavier ones. Consequently, to coherently evaluate the adaptive capabilities of UDR and ADR, we discarded this configuration and evaluated the subsequent tasks using the previous configuration, which perfectly demonstrates the agent’s sensitivity to the reality gap.

Observing the Domain Randomization results, it is noticeable that they perfectly confirm our theoretical expectations regarding Sim-to-Real transfer. Furthermore, they highlight a strong dependency of the learning quality on the width of the randomization range in the UDR technique.

Regarding UDR, the performance difference between the tested ranges highlights critical dynamics correlated to underfitting and distribution shift. Using a range of $[0.5, 4.0]$ kg, the agent is forced to handle heterogeneous physical dynamics (very light and heavy cubes) during the same training phase. This strong variance creates confusion in the neural network, which learns a conservative and unstable policy, leading the success rate to drop to 42.00%. In contrast, the $[0.5, 2.5]$ kg range proves to be optimal. It introduces enough variety to avoid overfitting on the 1kg cube, while remaining sufficiently bounded to allow the agent to learn a stable and adaptive policy able to manage the target domain, achieving a success rate of 100.00%. Expanding the boundaries to $[0.5, 10.0]$ kg, we again obtain a 100% success rate. However, this does not imply a real physical adaptation. First, the 5kg target is no longer Out-of-Distribution: it falls within the training bounds, transforming it into a standard, frequently encountered scenario

for the agent. Secondly, the 10kg upper bound forces the network to converge on the same brute-force approach, meaning the agent learns to apply the maximum available torque regardless of the context.

Finally, the results obtained by ADR are highly consistent with its adaptive nature, achieving a perfect 100.00% success rate. This outcome is coherent, since the ADR expands the mass variance only after the agent stabilizes its current performance, it overcomes the instability issue encountered with a wide UDR range. This demonstrates that, although UDR can match ADR’s performance, it requires a more expensive manual tuning of the randomization boundaries.

4.2. Methodological Critique

Although this study properly highlights Sim-to-Real transfer dynamics, it is necessary to address some methodological limitations.

Firstly, simulating the reality gap exclusively through mass variation represents a considerable simplification. In the real world, robotic implementations must face multiple discrepancies, including friction and sensor noise. Expanding the Domain Randomization to include a broader range of physical parameters would provide a more rigorous evaluation of policy robustness.

In addition, the emergence of brute-force approach reveals a limitation in the structure of the reward function. Even if applying maximum torques results in a high success rate, from an energetic point of view, it is significantly inefficient and potentially destructive for the physical hardware. To facilitate Sim-to-Real transfer, future iterations must incorporate energy penalties and regularize output actions, encouraging the network to learn smoother and more realistic movements.

5. Conclusion

This work demonstrates that off-policy algorithms are significantly more effective than on-policy methods such as PPO for continuous robotic manipulation tasks, achieving a success rate of 74% instead of PPO’s 30%. In the Hopper environment, the Actor-Critic architecture proved to be more stable and twice as fast to converge as REINFORCE. As for Sim-to-Real transfer, Domain Randomization effectively helps overcome the reality gap: both UDR with a well-calibrated range of $[0.5, 2.5]$ kg and ADR achieve a 100% success rate in the target domain, with ADR eliminating the need for manual parameter tuning. However, a key limitation emerges in several configurations: when the randomization ranges are too wide or the reward signal is not sufficiently constrained, agents tend to adopt brute-force approaches rather than learning physically correct behaviors. Future work should extend Domain Randomization to more physical parameters and add energy penalties to the reward function for smoother and safer real-world deployment.

6. Acknowledgments

6.1. Project Report

The work of this report is the result of personal analysis, without the use of generative AI systems. The entire content has been written by ourselves, only using dictionaries when necessary to search for unknown English words.

6.2. Project Code

- **GenAI Systems Used:** Gemini (3.1 Pro version), Claude (4.6 Sonnet version).
- **Publishers:** Google (Gemini); Anthropic (Claude).
- **Description of use:** The tools acted as interactive assistants in the process of general debugging, syntax checking, and correction of some trivial problems while working on Python scripts development. Additionally, they were used to accelerate the hyperparameter tuning of the PPO and SAC algorithms by excluding ineffective combinations and to assist in identifying key insights from the scientific literature mentioned in the assignment.
- **Human supervision:** All code snippets developed with the help of AI as well as the process of hyperparameters tuning were thoroughly analysed, tested, and verified empirically. All decisions related to engineering reasoning, including those of the optimal choice of hyperparameters, algorithms' logic, domain randomization policies, and analytical approach, were taken only by humans.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An introduction (Second Edition)*, 2018. [1](#)
- [2] J. Kober, J. Bagnell, & J. Peters, *Reinforcement learning in robotics: A survey*, The International Journal of Robotics Research, 2013.
- [3] P. Kormushev, S. Calinon, & D. G. Caldwell, *Reinforcement learning in robotics: Applications and real-world challenges*, 2013.
- [4] S. Höfer et al., *Perspectives on sim2real transfer for robotics: A summary of the R: SS 2020 workshop*, 2020. [2](#)
- [5] J. Tobin et al., *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*, arXiv, 2017. [2](#)
- [6] X. B. Peng et al., *Sim-to-real transfer of robotic control with dynamics randomization*, 2018.
- [7] T. Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, ICML, 2018. [1](#)
- [8] J. Schulman et al., *Proximal policy optimization algorithms*, arXiv, 2017. [1](#)
- [9] F. Muratore et al., *Robot learning from randomized simulations: A review*, Frontiers in Robotics and AI, 2022.